DAA LAB 11

Kshitij Kejriwal-252010005

1st April 2026

Dijkstra's Algorithm

1 Code:

```python
import heapq

class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.graph = {i: [] for i in range(vertices)}

    def add_edge(self, u, v, w):
        self.graph[u].append((v, w))

    def dijkstra(self, src):
        dist = [float('inf')] * self.V
        dist[src] = 0

        pq = []
        heapq.heappush(pq, (0, src))

        while pq:
            current_dist, u = heapq.heappop(pq)

            for v, weight in self.graph[u]:
                if dist[v] > dist[u] + weight:
                    dist[v] = dist[u] + weight
                    heapq.heappush(pq, (dist[v], v))

        return dist


# -------- USER INPUT --------
n = int(input("Enter number of vertices: "))
m = int(input("Enter number of edges: "))

```

```python
g = Graph(n)

print("Enter edges (u v weight):")
for _ in range(m):
    u, v, w = map(int, input().split())
    g.add_edge(u, v, w)

src = int(input("Enter source node: "))

# -------- DIJKSTRA --------
distances = g.dijkstra(src)

# -------- OUTPUT --------
print("\nShortest distances from source node:", src)
for i in range(len(distances)):
    print(f"{src} -> {i} = {distances[i]}")
```

2 Output:

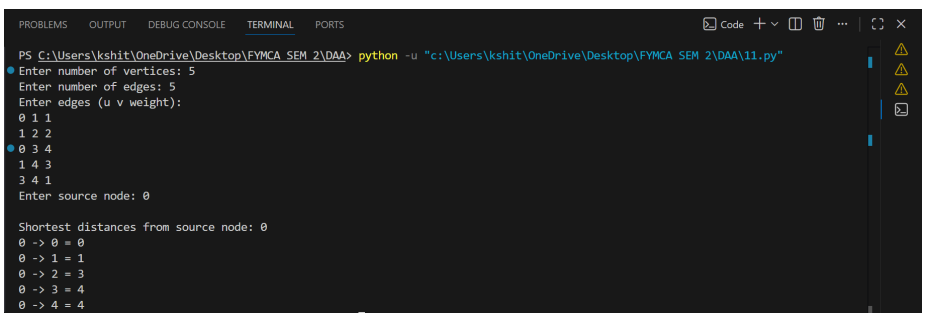


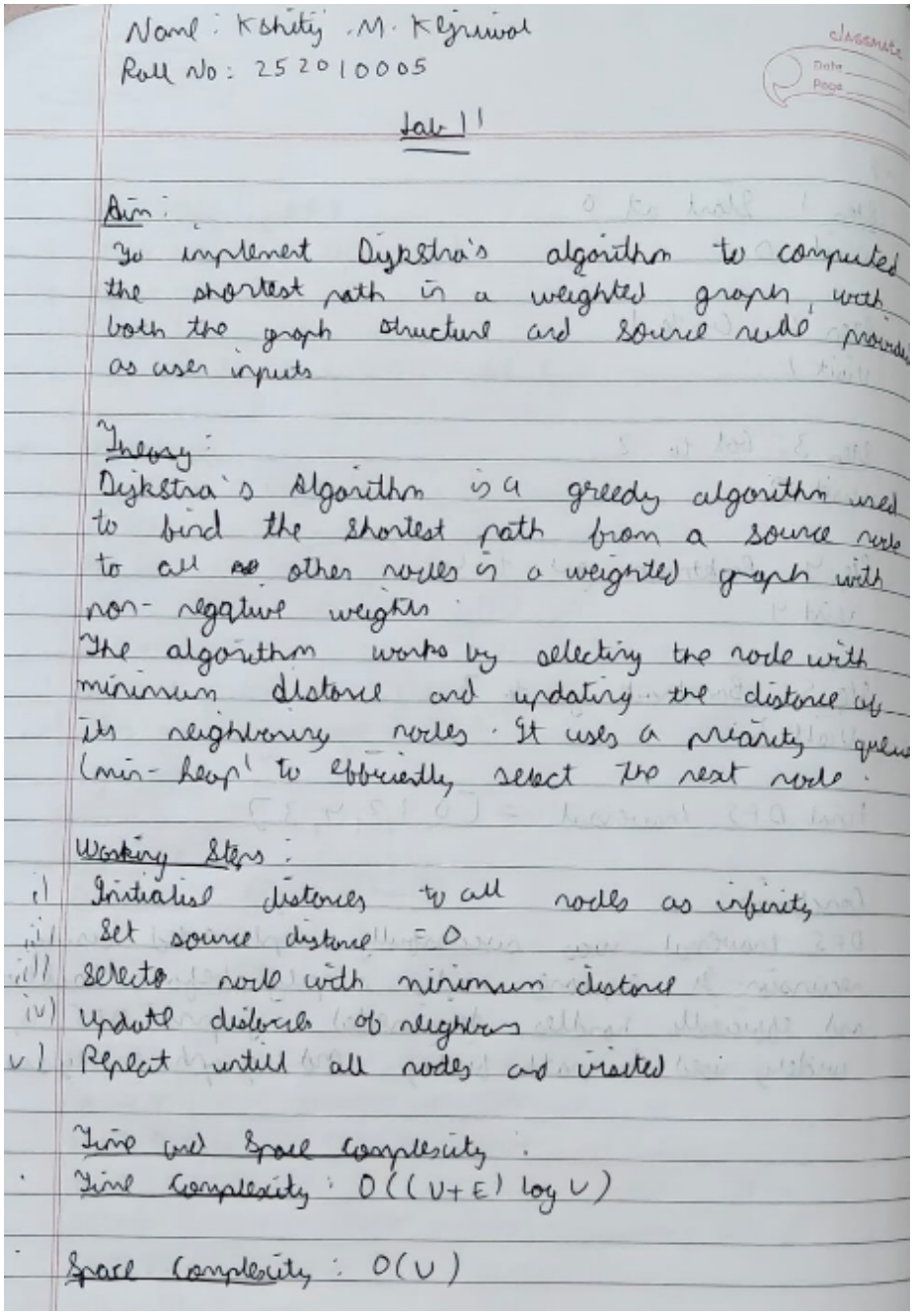
Figure 1: Output

3 Pseudocode:

Algorithm 1 Dijkstra's Algorithm

```
procedure DIJKSTRA(G, source)
    Initialize distance array with ∞
    dist[source] ← 0
    Insert source into priority queue
    while priority queue is not empty do
        u ← node with minimum distance
        for each neighbor v of u do
            if dist[v] > dist[u] + weight(u, v) then
                dist[v] ← dist[u] + weight(u, v)
                Update priority queue
            end if
        end for
    end while
end procedure
```

4 Explanation:



Name: Kshitij M. Kejriwal
Roll No: 252010005

Lab 11

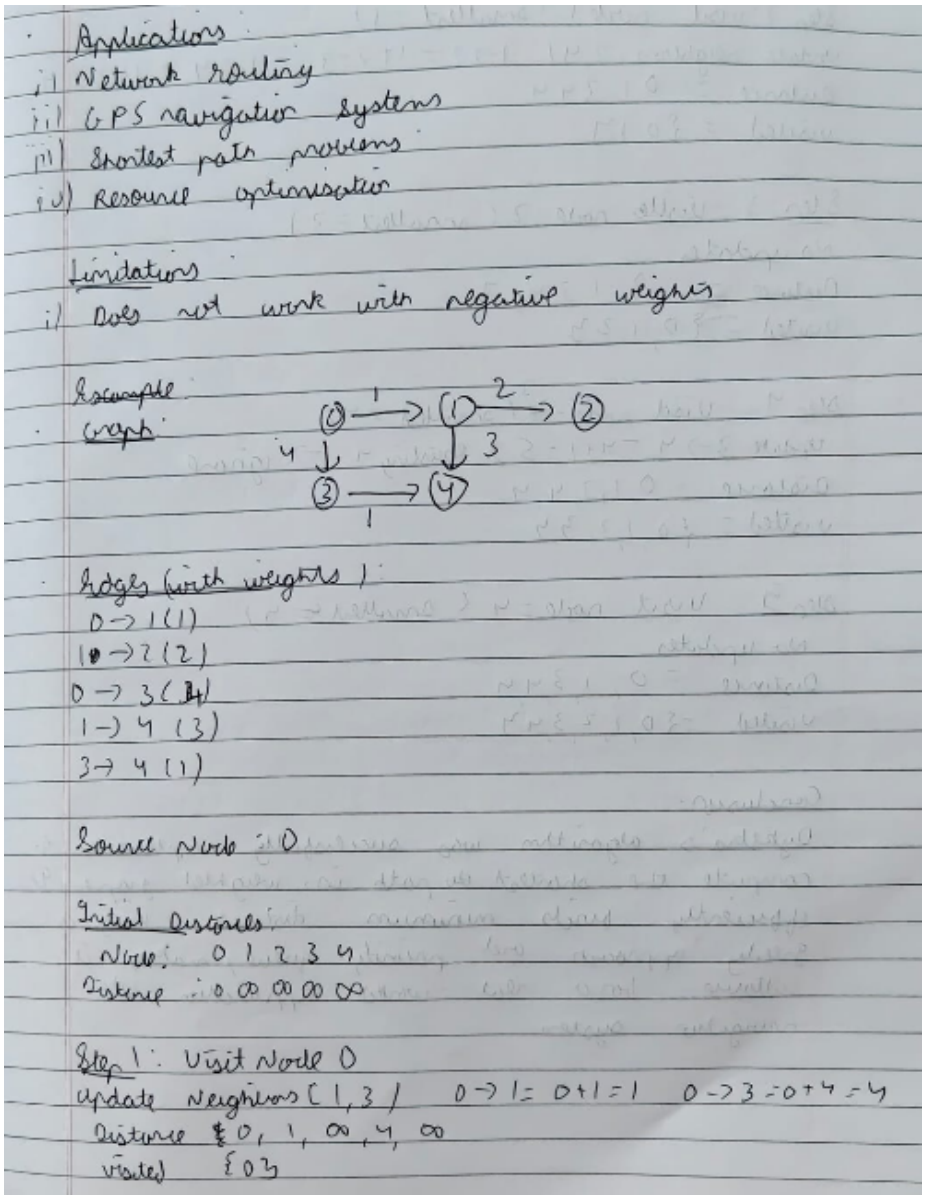
Aim:
To implement Dijkstra's algorithm to computed the shortest path in a weighted graph, with both the graph structure and source node provided as user inputs

Theory:
Dijkstra's Algorithm is a greedy algorithm used to find the shortest path from a source node to all other nodes in a weighted graph with non-negative weights.
The algorithm works by selecting the node with minimum distance and updating the distance of its neighbouring nodes. It uses a priority queue (min-heap) to efficiently select the next node.

Working Steps:
i) Initialise distances to all nodes as infinity
ii) Set source distance = 0
iii) Select node with minimum distance
iv) Update distances of neighbours
v) Repeat until all nodes are visited

Time and Space Complexity:
- Time Complexity: $O((V+E)\log V)$
- Space Complexity: $O(V)$

Figure 2: Explanation



Applications:

i) Network routing
ii) GPS navigation systems
iii) Shortest path problems
iv) Resource optimisation

Limitations:

i) Does not work with negative weights

Example:

Graph:

Edges (with weights):

0 → 1 (1)
0 → 2 (2)
0 → 3 (4)
1 → 4 (3)
3 → 4 (1)

Source Node: 0

Initial Distances:

Node: 0 1 2 3 4
Distance: 0 ∞ ∞ ∞ ∞

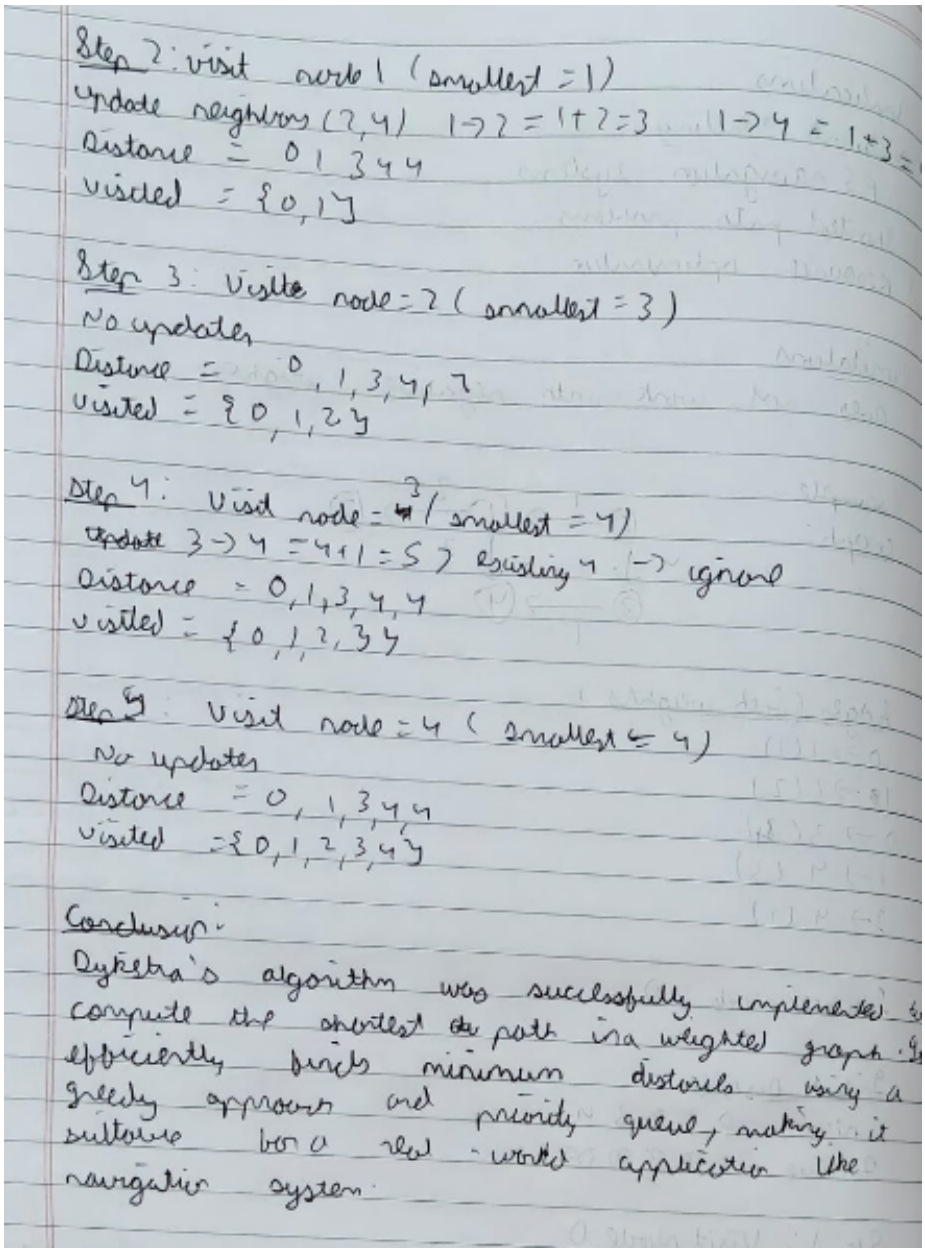
Step 1: Visit Node 0

Update Neighbors (1, 3) 0 → 1 = 0+1 = 1 0 → 3 = 0+4 = 4

Distance {0, 1, ∞, 4, ∞}

Visited {0}

Figure 3: Explanation



Step 2: visit node 1 (smallest = 1)
Update neighbors (2, 4) 1 -> 2 = 1+2=3 1 -> 4 = 1+3=
Distance = 0 1 3 4 4
visited = {0, 1}

Step 3: Visite node = 2 (smallest = 3)
No updates
Distance = 0, 1, 3, 4, 7
visited = {0, 1, 2}

Step 4: Visit node = 3 (smallest = 4)
Update 3 -> 4 = 4+1 = 5 > existing 4 -> ignore
Distance = 0, 1, 3, 4, 4
visited = {0, 1, 2, 3}

Step 5: Visit node = 4 (smallest = 4)
No updates
Distance = 0, 1, 3, 4, 4
visited = {0, 1, 2, 3, 4}

Conclusion:
Dijkstra's algorithm was successfully implemented to compute the shortest path in a weighted graph. It efficiently finds minimum distances using a greedy approach and priority queue, making it suitable for real-world applications like navigation system.

Figure 4: Explanation